RunPod Serverless Case Study

Bruno Diniz, bruno.reisdiniz@gmail.com, 8 September 2026

GitHub: https://github.com/thekneezee/flux-runpod-serverless
Live demo: https://flux-serverless-demo.onrender.com
Login: reviewer / runpod2026

The demo is on a free host that sleeps when nobody visits, so the first page load takes about 50 seconds and the first picture about 30 seconds more while a graphics card starts up. After that pictures come back in seconds. The "Pre-warm the GPU" button gets the card ready before you type.

What I built

I deployed FLUX.1-dev, an AI model that turns text into pictures, as a serverless endpoint on RunPod. Then I built a chat page where you type a sentence and get a picture back, so the model can actually be used and not just described.

The model is about 34 gigabytes and runs on a rented 48 gigabyte graphics card. The card only switches on when someone asks for a picture and switches off five seconds later, so when nobody is using it the cost is zero. I built, tested and measured the whole thing for under one dollar of the thirty dollar credit.

The interface

Making a picture takes about 16 seconds, and most apps hide that behind a spinning circle. Mine does not. The code on the graphics card sends an update after every one of the 28 rounds of work it does, so you watch the progress bar move from 1 of 28 up to 28 of 28, and then the picture appears with the details underneath: the time it took, which graphics card ran it, and what it cost.

The page also shows what the endpoint is doing right now, so you can watch a graphics card switch on and off again. There is a running total of what you have spent, a cancel button, and a practice mode that fakes everything so anyone can look around without a password or any cost.

Results measured on the live endpoint

- About 16 seconds for one picture at 1024 by 1024 with 28 steps
- About 0.8 of a cent per picture
- Zero cost when nobody is using it
- First picture after a quiet period takes about 22 seconds extra
- 23 requests sent, 23 completed, none failed
- Under 1 dollar spent in total

One decision worth explaining

The instructions suggested putting the model inside the Docker container. I did not, and I think that was the right call. RunPod builds containers with a hard 30 minute limit and downloading 34 gigabytes will not finish in that time. On top of that, this model needs a password to download and RunPod's build system has no safe way to supply one, so the only option would be writing my password into a public file.

RunPod's own documentation recommends their cached model feature for exactly this situation, so I used it. They copy the model onto the machine in advance. That copy took 28 minutes, happened once, and was not charged. My container ended up 12 gigabytes instead of 46 and the model loads in 7 seconds. I also wrote a backup that downloads it if the cache is ever missing.

What is in the repository

The code for the graphics card, the chat page, 44 automated tests that run without a graphics card, scripts that test and measure the live endpoint, a Postman collection, a short video of it working, and screenshots. There is also a file called EXPLAINED.md that walks through the whole project in plain language, including the four problems I hit and how I solved them.

Happy to walk through any part of it.